NEXUSGEAR

PRESENTATION 01

Team, Setup & Planning

E-Commerce Project

A curated home for high-performance gaming peripherals and ergonomic desk setups.

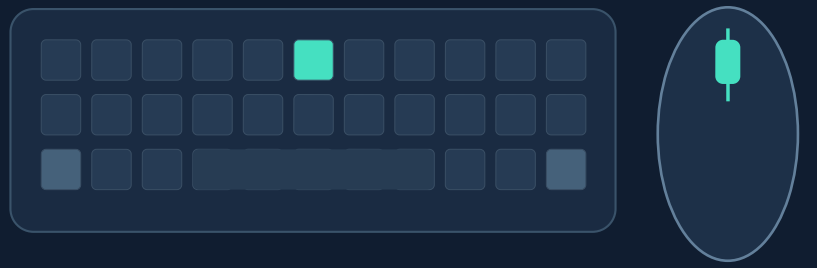
Popa-Ion Andrei
Frontend Lead & Customer Experience

Șerban Alexandru-Cristian-George
Backend Lead & System Integrations

Popa-Ion Andrei: UI, client flows and responsive design
Șerban Alexandru-Cristian-George: architecture, database, Stripe and security

THE NICHE

Build a better desk experience.



30 seeded products / 4 categories
GitHub • custom stack • sandbox payments

Context & store niche

Curated gear for people who spend serious time at their desk.

NexusGear

A focused store, not a generic mega-marketplace.

TARGET CUSTOMER

Gamers
Precision & performance

Programmers
Comfort for long sessions

Remote workers
A healthier desk setup

CATALOG SCOPE

30 products across 4 categories

SEED SCRIPT

Mechanical keyboards	60%, 75% & TKL layouts; hot-swappable switches	8
Gaming mice	Ultralight wired/wireless; high-DPI optical sensors	8
Deskmats & mousepads	Extended XL/XXL cloth and hybrid glide pads	8
Ergonomic gaming chairs	Lumbar support, 4D armrests, breathable fabric	6

Every seeded item will include an image, price and stock level.

Competitor analysis

Three real stores in the same niche: borrow proven patterns, differentiate on simplicity and breadth.

PC Garage

ROMANIA

ADOPT

Technical detail & attribute filters: switch type, DPI and connection mode.

DO DIFFERENTLY

A calm, uncluttered UI without aggressive promo banners; a more intuitive checkout.

QwertyKey

ROMANIA

ADOPT

Clean product showcases and transparent stock-availability indicators.

DO DIFFERENTLY

A complete desk setup in one order: keyboards, mice, pads and chairs.

Secretlab

INTERNATIONAL

ADOPT

Clear ergonomic explanations and material-quality breakdowns on product pages.

DO DIFFERENTLY

A custom-built admin backend for agile fulfilment and instant stock tracking.

Positioning: technical confidence + ergonomic context + a quieter, easier buying journey.

Technology stack & rationale

A deliberately simple, custom-built stack that supports the mandatory A1–A12 requirements.

FRONTEND

Bootstrap 5 + Bootstrap Icons

Mobile-first responsive layout with accessible, reusable components and no separate client-framework build complexity.

A11

BACKEND

ASP.NET Core (.NET 10)

A strongly typed, high-throughput web backend with server-side validation, antiforgery protection and authentication support.

A4 · A10

DATABASE

SQLite + Entity Framework Core

A lightweight relational store with migrations and straightforward local setup, well suited to deterministic seed data.

A1

PAYMENTS

Stripe — Test Mode

Enables safe sandbox card testing, with order status changed to Paid only after a verified server-side webhook.

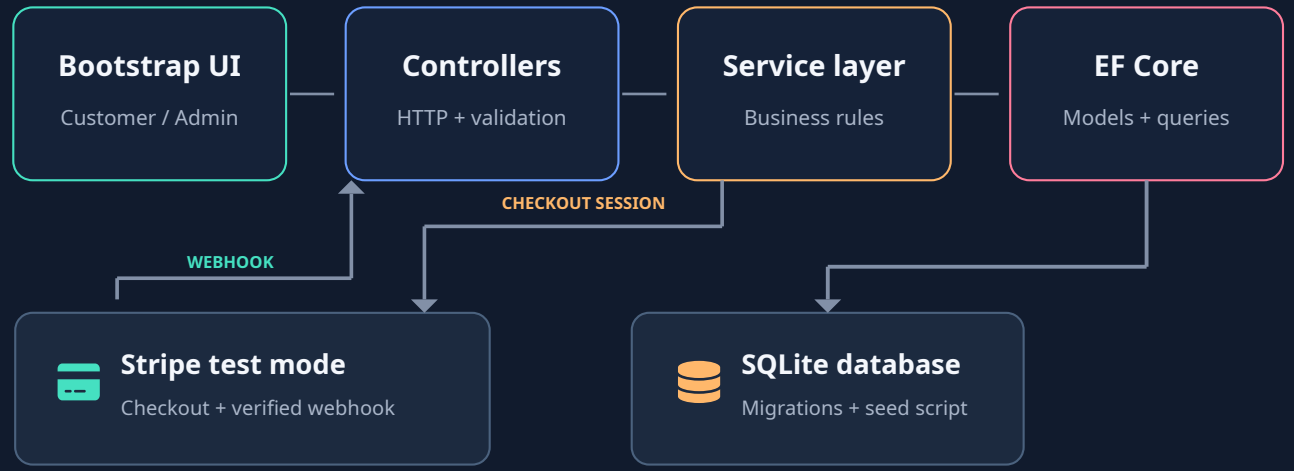
A6

A4 implementation note The spec explicitly requires bcrypt or Argon2 password storage; configure Identity to meet that criterion.

High-level architecture & repository setup

Monolithic ASP.NET Core application, layered for clear ownership and easy local execution.

REQUEST / DATA FLOW



One deployable application; clear internal layers instead of a separate client API stack.

GitHub workflow

- Protect main; build in feature branches.
- Examples: feature/cart, feature/auth.
- Review pull requests; tag milestone releases.

Local execution

README-guided start in ≤5 commands; use dotnet run.

Secrets & environment

Keep keys in local .env / appsettings.Development.json; commit only a sanitized .env.example.

Responsibility matrix: A1–A12

Every mandatory requirement has one named primary owner; A12 is shared by both team members.

A1	**Product catalog** — Paginated listing (≤24/page), category views, product details and seed data.	Popa-Ion Andrei
A2	**Basic search** — Case-insensitive product-title search with a clear empty state.	Popa-Ion Andrei
A3	**Shopping cart** — Server-calculated totals, stock-boundary checks and cart persistence.	Șerban Alexandru-Cristian-George
A4	**User accounts** — Registration/login, saved address and spec-compliant password hashing.	Șerban Alexandru-Cristian-George
A5	**Checkout flow** — Address, 2 shipping methods, order review and confirmation.	Popa-Ion Andrei
A6	**Sandbox payment** — Stripe test session; verified webhook is the source of Paid status.	Șerban Alexandru-Cristian-George
A7	**Stock management** — Concurrency-safe deduction on paid webhook; restock on cancellation.	Șerban Alexandru-Cristian-George
A8	**Order history** — Customer dashboard with past orders, line items and statuses.	Popa-Ion Andrei
A9	**Admin panel** — Custom product/category CRUD, image upload and order transitions.	Șerban Alexandru-Cristian-George
A10	**Basic security** — Server validation, CSRF, EF Core parameterization and admin guards.	Șerban Alexandru-Cristian-George
A11	**Responsive design** — Every customer-facing page usable down to a 360 px viewport.	Popa-Ion Andrei
A12	**Automated tests** — At least 10 xUnit tests for cart totals, stock logic and webhooks.	BOTH

Roadmap: Weeks 4–7 — foundation & core browsing

Build project structure, seed data, catalog, authentication and search before checkout and payment work.

WEEK 4

SETUP

Create ASP.NET Core structure.
Assigned: Șerban Alexandru-Cristian-George

DATA

SQLite migrations + seed script for 30 products / 4 categories.
Assigned: Șerban Alexandru-Cristian-George

UI

Build shared Bootstrap layout and navigation.
Assigned: Popa-Ion Andrei

WEEK 5

A1

Implement catalog, pagination and category views.
Assigned: Popa-Ion Andrei

A4

Implement registration, login/logout and password hashing.
Assigned: Șerban Alexandru-Cristian-George

WEEK 6

DATA MODEL

Define product, cart, order and payment entities.
Assigned: Șerban Alexandru-Cristian-George

A2

Implement case-insensitive product search.
Assigned: Popa-Ion Andrei

WEEK 7

INTEGRATE

Connect the catalog to seeded SQLite data.
Assigned: Șerban Alexandru-Cristian-George

VALIDATE

Check category pages, empty-search feedback and login.
Assigned: Popa-Ion Andrei

CHECKPOINT

Review Weeks 4–7 progress and confirm the next build tasks.
Assigned: Popa-Ion Andrei

Week 7 checkpoint • Seeded catalog • Category navigation • Search empty state • Login flow

Roadmap: Stage 3 — build, test & deliver

Weeks 8–12 • Milestone P3: final functional delivery and live demonstration.

WEEK 8

A3

Server-side cart + stock validation.

Assigned: Șerban Alexandru-Cristian-George

A5

Checkout wizard + delivery options + order summary.

Assigned: Popa-Ion Andrei

WEEK 9

A6

Stripe Checkout + webhook confirmation.

Assigned: Șerban Alexandru-Cristian-George

A7

Concurrency-safe inventory deduction + rollback.

Assigned: Șerban Alexandru-Cristian-George

WEEK 10

A9

Custom admin CRUD + order management.

Assigned: Șerban Alexandru-Cristian-George

A8

Customer order history page.

Assigned: Popa-Ion Andrei

WEEK 11

A12

Automate ≥10 tests; run with dotnet test.

Assigned: Popa-Ion Andrei & Șerban Alexandru-Cristian-George

A10 / A11

Security checks + 360 px viewport audit.

Assigned: Popa-Ion Andrei & Șerban Alexandru-Cristian-George

WEEK 12

DOCS

Finalize README and test instructions.

Assigned: Popa-Ion Andrei

BACKUP

Record a backup demo.

Assigned: Șerban Alexandru-Cristian-George

RELEASE

Tag P3 and submit final deliverables.

Assigned: Șerban Alexandru-Cristian-George

P3 demo path: register → search → cart → test payment → webhook confirmation → stock decrease → admin order update.

Risk management & mitigations

Three delivery risks with practical controls built into the implementation plan.

01 Inventory race conditions

A7 · STOCK MANAGEMENT

IMPACT

Two customers could try to buy the final unit at the same time, causing negative stock.

MITIGATION

Use atomic database transactions and EF Core concurrency controls when processing the confirmed payment webhook.

02 Stripe webhook delivery fails locally

A6 · SANDBOX PAYMENT

IMPACT

Orders may remain Unpaid if Stripe cannot reach the local development machine.

MITIGATION

Use the official Stripe CLI from Week 8 to forward events to the local webhook endpoint.

```
stripe listen --forward-to localhost:5000/api/webhook
```

03 Scope creep delays testing

A12 · AUTOMATED TESTS

IMPACT

The required 10 tests may be incomplete or fail during the live assessment.

MITIGATION

Write xUnit tests alongside the cart and order-processing services instead of postponing testing until Week 11.

Risk ownership is shared across delivery: build observability and tests while the relevant feature is being implemented.

Deliverables checklist & next steps

A submission-ready repository with a clear path from foundation work to final delivery.

P1 READINESS CHECKLIST

- [x] GitHub repository created; teaching staff added as collaborators.
- [x] Initial README committed with setup, run and environment instructions.
- [x] A sanitized .env.example is tracked; secrets and Stripe test keys stay local.
- [x] Each student commits their own work so contribution is visible in history.
- [x] P1 commit tagged and slides/repository link submitted at least 24 hours before defense.
- [x] README declares any AI tools used, as required by the course specification.



Next: implementation

Core features & integrations

- Build out the catalog and secure account flows.
- Add the cart, checkout, sandbox payment and stock controls.
- Complete administration, order history, tests and responsive checks.

QUESTIONS & FEEDBACK

NexusGear • P1